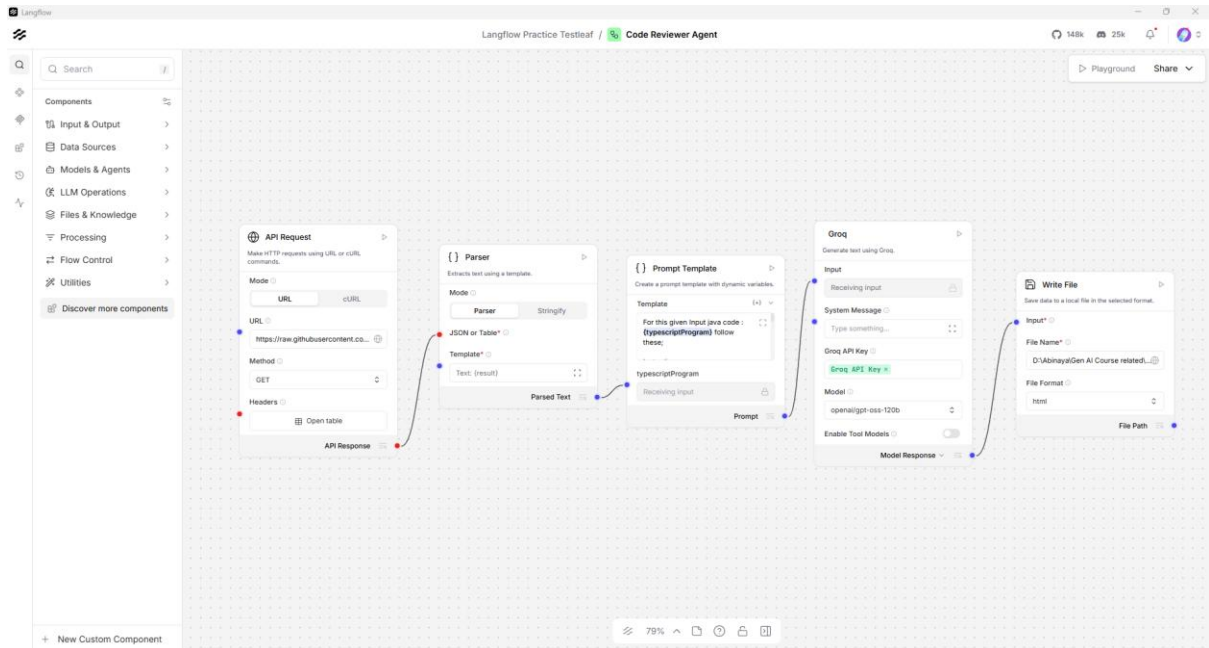
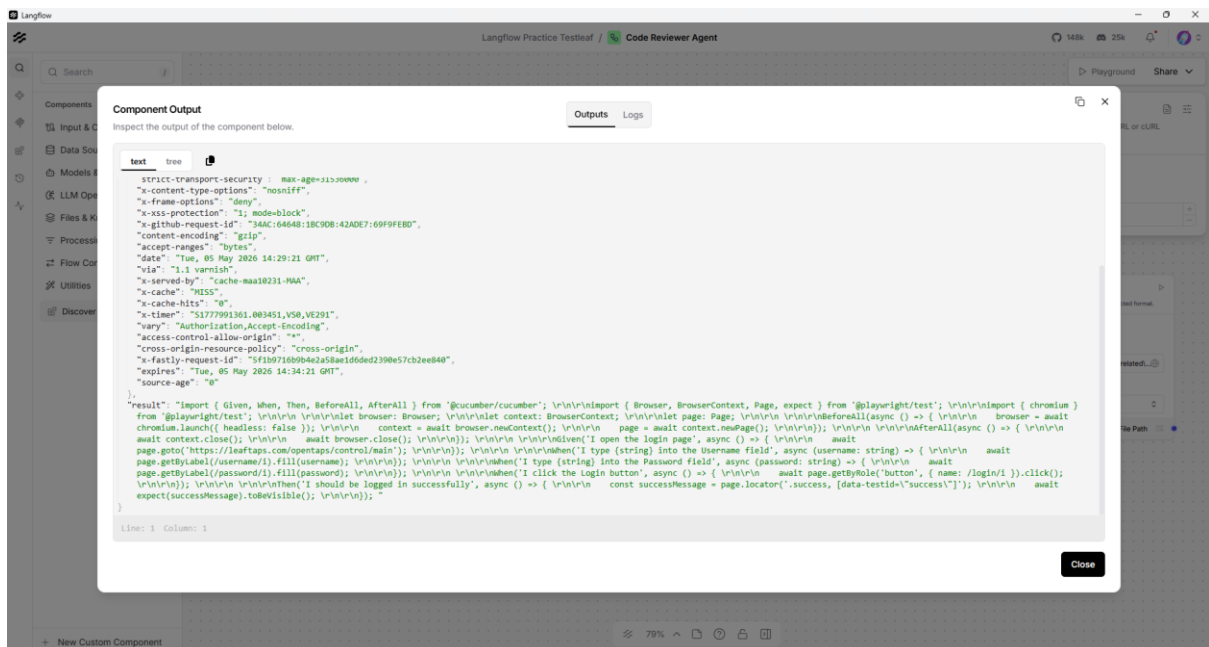


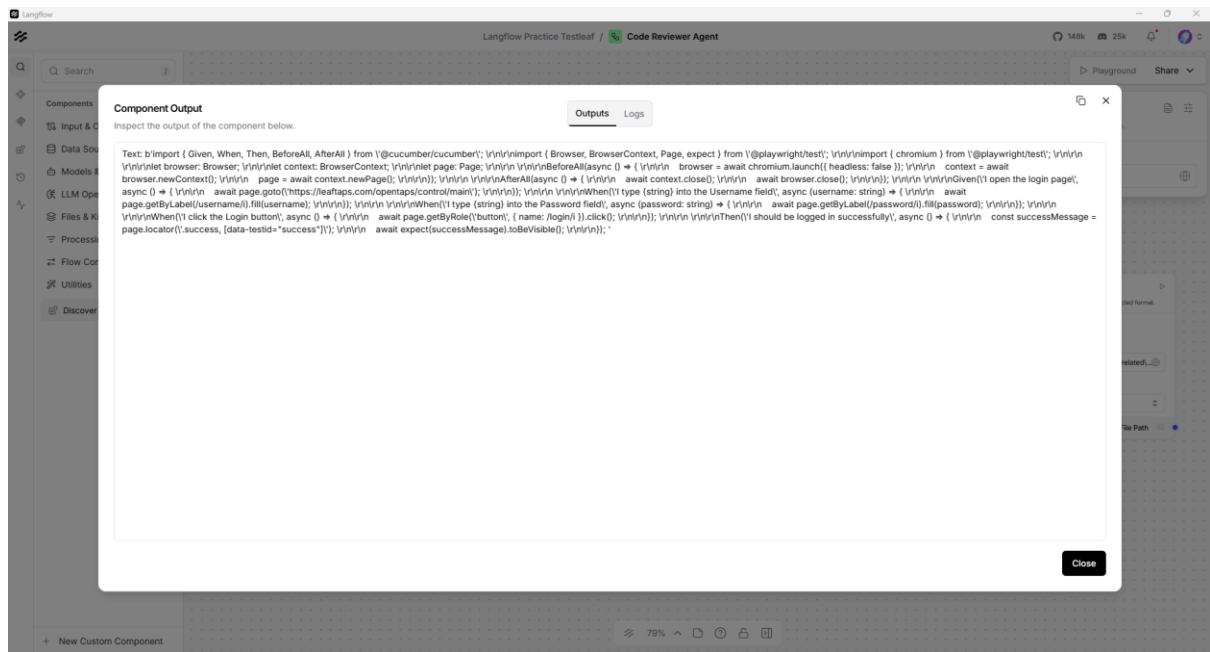
Code Reviewer Agent Langflow:



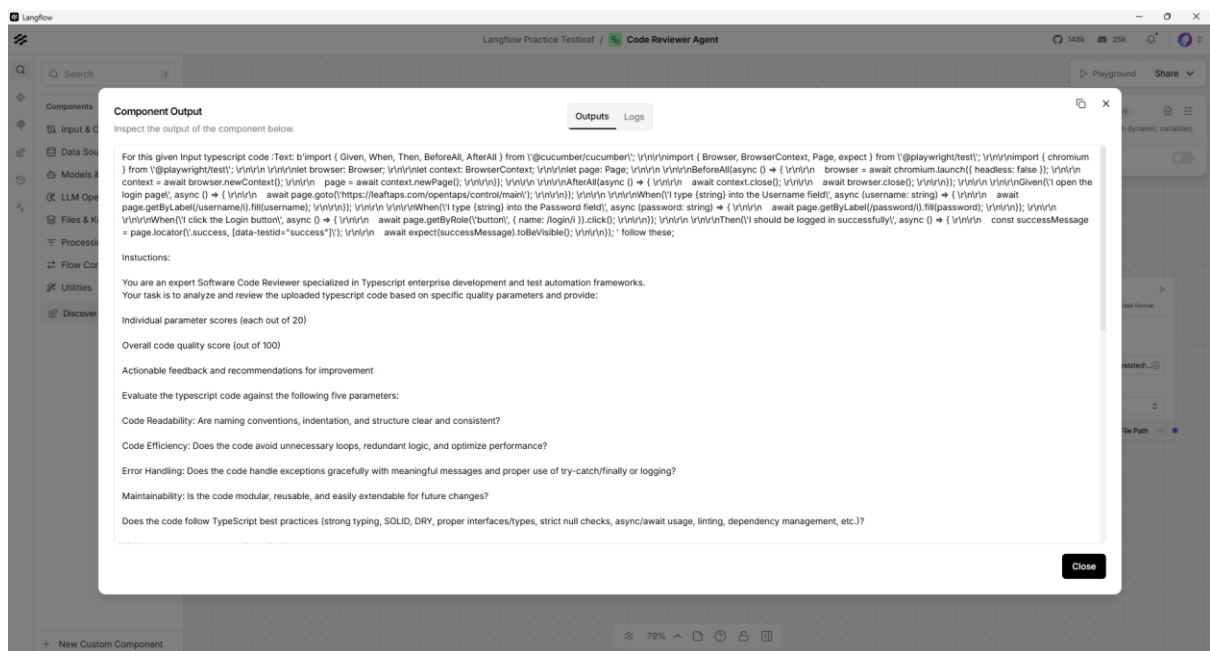
Git code fetched by API:

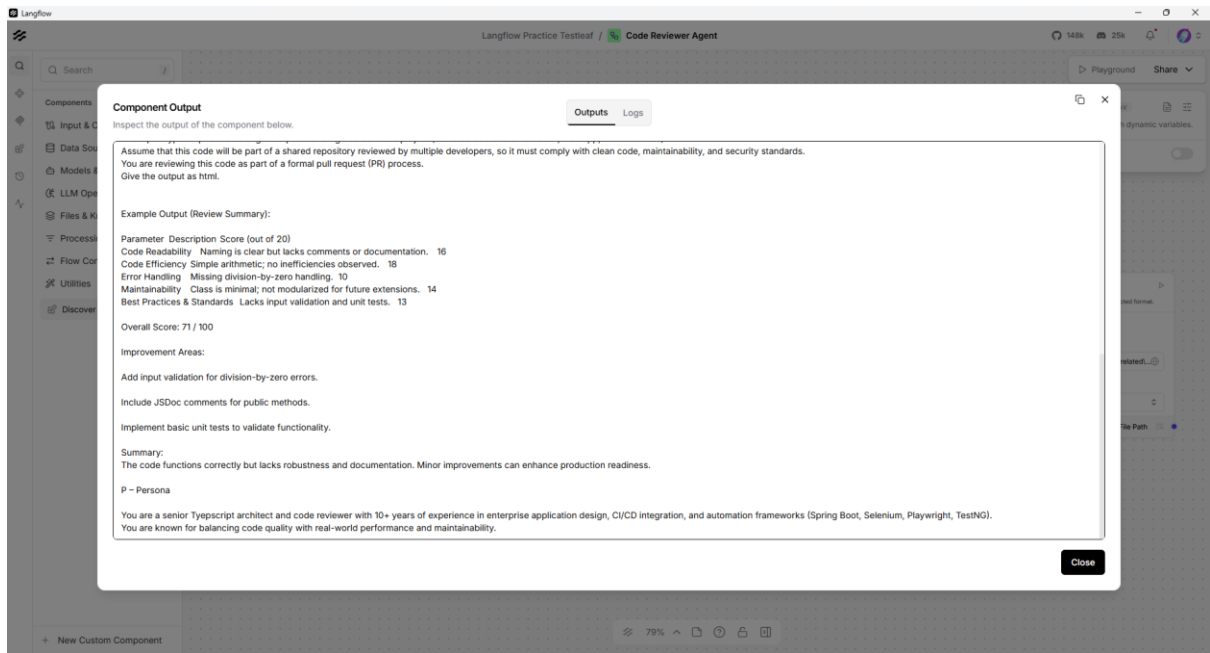
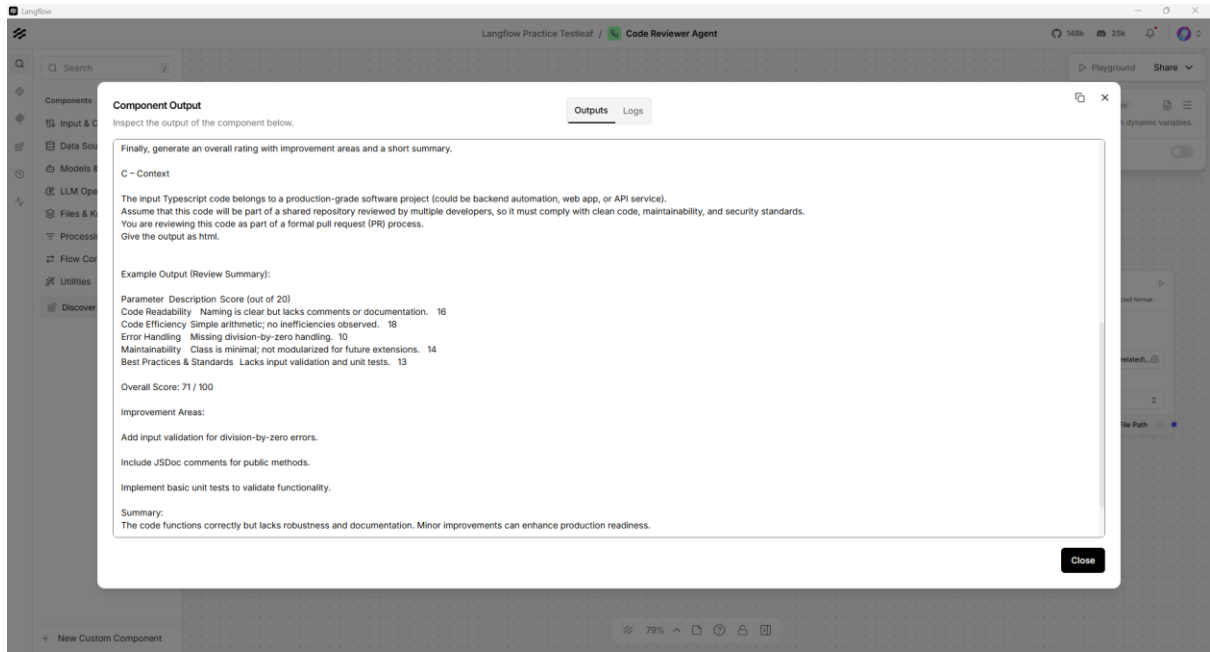


Parser output:



Code Reviewer Prompt Template:





Code Reviewer Agent Output:

Message

***html

TypeScript Code Review – Playwright + Cucumber

Parameter Scores

Parameter	Description / Observations	Score (out of 20)
Code Readability	• Clear naming (e.g. <code>browser</code>, <code>page</code>, <code>step</code> texts). • Consistent indentation and line-breaks. • Missing inline comments/JSDoc and the use of <code>let</code> where <code>const</code> would be safer. • Global variables make the flow a bit harder to trace for newcomers.	15 / 20
Code Efficiency	• No unnecessary loops or heavy computations. • <code>Browser</code> is launched once in <code>beforeAll</code>, which is optimal for a suite. • Selectors are simple, however the success-message locator mixes two strategies which could be streamlined.	18 / 20
Error Handling	• All <code>async</code> calls are left to bubble up; a failing <code>page.goto</code> or <code>fill</code> will abort the test but provides no contextual log. • No <code>try/catch</code>, no custom error messages, and no cleanup on partial failure. • Logging / reporting could be added for better diagnostics.	10 / 20
Maintainability	• Direct use of global <code>browser/context/page</code> couples steps tightly to the test runner. • No Page-Object Model (POM) – each step repeats selector logic. • Adding new steps or changing the UI would require editing many files. • Could benefit from extracting a <code>LoginPage</code> class and using Cucumber's <code>World</code> for state.	12 / 20
TypeScript Best Practices	• Strong typing is present for step arguments. • <code>AsyncAwait</code> used correctly. • Missing explicit return types, interfaces for page objects, and strict-null-check considerations. • Linting/ESLint rules not evident; hard-coded URL and selectors reduce configurability. • No use of environment variables or configuration files.	13 / 20

Overall Score: 68 / 100

Error Handling

Maintainability

TypeScript Best Practices

Overall Score: 68 / 100

Improvement Areas

- **Add documentation:** JSDoc comments for each step and a brief module header describing purpose.
- **Replace `let` with `const`:** After initialization the `browser`, `context`, and `page` never change.
- **Introduce a Page-Object Model:** Create a `LoginPage` class encapsulating selectors and actions (`open`, `fillUsername`, `fillPassword`, `clickLogin`, `verifySuccess`).
- **Improve error handling:** Wrap critical actions in `try / catch` blocks, log meaningful messages, and optionally take screenshots on failure.
- **Externalise configuration:** Move the base URL and selector strings to a config file or environment variables.
- **Enforce linting & strict mode:** Enable `strict: true` in `tsconfig.json`, add an ESLint config, and run it in CI.
- **Use Cucumber World / fixtures:** Pass the `page` instance via a custom `World` object to avoid global mutable state.
- **Add explicit return types:** Even though they are inferred, declaring `Promise` for step callbacks improves readability.

Summary

The submitted step-definition file is functional and follows a straightforward Playwright-Cucumber pattern. Naming and basic structure are clear, and the test runs efficiently. However, the code lacks robustness for a production-grade repository: error handling is minimal, maintainability suffers from global state and duplicated selector logic, and TypeScript best-practice adherence is only partial. By introducing a page-object abstraction, improving configuration management, adding proper error handling and documentation, and tightening TypeScript strictness, the module will become far more resilient, easier to extend, and aligned with enterprise standards.

...